Introductory Programming and Problem-Solving Skills

Iterative Control

Week - 4

Basanta Singh

Let’s Recap

Navigation

Let’s Recap

- Boolean Expressions
- Relational Operators
- Logical Operators
- Operator Precedence
- Use of Parentheses
- Sequential Flow
- Selection Control – if, if-else, elif, nested conditions.

Control Structures (continue)

1. Multiple Messages

Navigation

1. Multiple Messages

```
print("Hello Class")
print("How Are You Doing?")
print("Ready With Note Copy?")
print("Let's Start Class!!")
```

Output:

```
Hello Class
How are you doing?
Ready With Note Copy?
Let's start Class!!
```

Correct Way?

Navigation

1. Multiple Messages
2. Correct way to print

But what about when we have to print same message for several times?

```
print("Hello World")
print("Hello World")
print("Hello World")
print("Hello World")
print("Hello World")
print("Hello World")
print("Hello World")
print("Hello World")
print("Hello World")
print("Hello World")
```

Correct Way?

Nothing To Worry!!

Navigation

1. Multiple Messages
2. Correct way to print
3. Solution for previous problem

Code Might Be Too Long Doing so.

Don’t worry to code long codes

Because

Iteration (or looping) is there for you to reduce code line

Before Going Further Let’s Discuss Operator That Will Be Used In Iterations

4.Membership Operator

Navigation

1. Multiple Messages
2. Correct way to print
3. Solution for previous problem
4. Membership Operator

- Python provides a convenient pair of **membership operators**. These operators can be used to easily determine if a particular value occurs within a specified list of values.

Membership Operators	Examples	Result
in	10 in (10, 20, 30)	True
	red in ('red','green','blue')	True
not in	10 not in (10, 20, 30)	False

4. Membership Operator

Navigation

1. Multiple Messages
2. Correct way to print
3. Solution for previous problem
4. Membership Operator continued

- The membership operator can be also used to check if a given **string occurs** within another:

```
>>> 'Dr.' in 'Dr. Madison'
True
```

Membership operators can be used to construct **Boolean expressions**.

4. Membership Operator

Navigation

1. Multiple Messages
2. Correct way to print
3. Solution for previous problem
4. Membership Operator continued


```
10 in (40, 20, 10):
        True
10 not in (40, 20, 10):
        False
.25 in (.45, .25, .65):
        True
grade = 'A', grade in ('A', 'B', 'C', 'D', 'F'):
        True
city = 'Houston', city in ('NY', 'Baltimore', 'LA'):
        False
```

Iterative Control

5.Iterative Control

Navigation

1. Multiple Messages
2. Correct way to print
3. Solution for previous problem
4. Membership Operator
5. Iterative Control

- An **iterative control statement** is a control statement providing the repeated execution of a set of instructions.

- An *iterative control structure* is a set of instructions and the iterative control statement(s) controlling their execution. Because of their repeated execution, iterative control structures are commonly referred to as “loops.”

In this lecture, we will be discussing about various types of loops and their execution time.

5.1.While Statement

Navigation

1. Multiple Messages
2. Correct way to print
3. Solution for previous problem
4. Membership Operator
5. Iterative Control

5.1. While Statement

- A **while statement** is an iterative control statement that repeatedly executes a set of statements based on a provided Boolean expression (condition).
- All iterative control needed in a program can be achieved by use of the while statement.

Syntax:

```
while(condition):
    #code to be executed
    Increment/ decrement
```

Description:

- **Condition: Here, the expression will be tested. The loop will be executed until the condition is false**
- **Increment/Decrement: Inorder increase and decrease value or expression the variable should be increment/decrement**

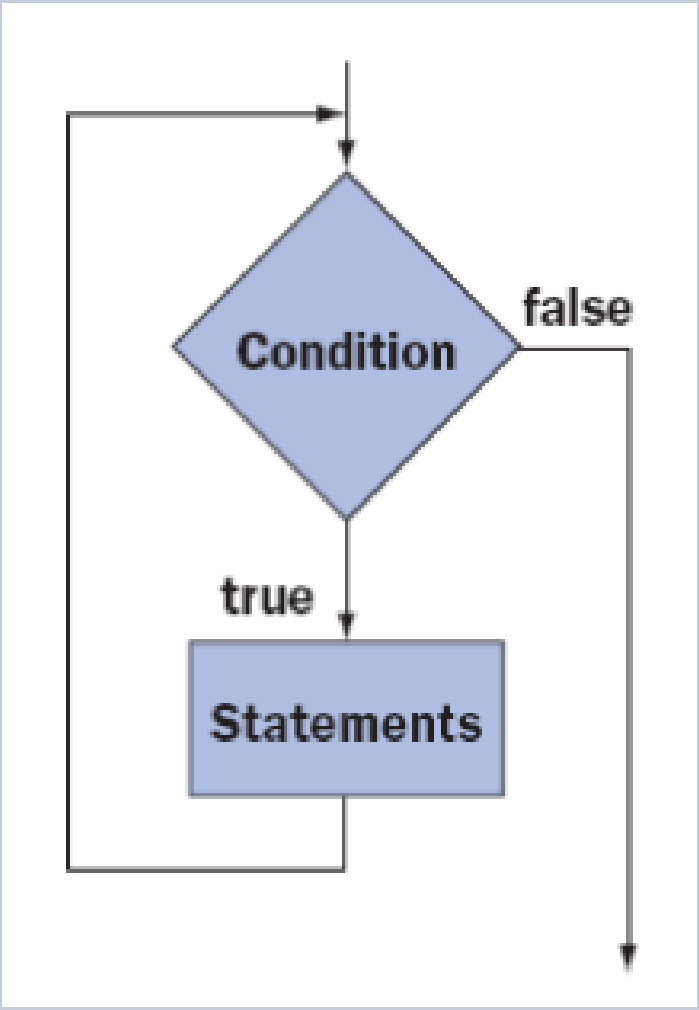
Flowchart

Navigation

1. Multiple Messages
2. Correct way to print
3. Solution for previous problem
4. Membership Operator
5. Iterative Control

5.1. While Statement

5.1.1. Flowchart for while loop statement




```
i = 1
while (i <= 10):
    print(i, "Hello World")
    i = i + 1
```

Explanation: As long as the condition of a while statement is true, the statements within the loop are (re)executed. Once the condition becomes false, the iteration terminates and control continues with the first statement after the while loop.

Let’s Visualize!!

Navigation

1. Multiple Messages
2. Correct way to print
3. Solution for previous problem
4. Membership Operator
5. Iterative Control
5.1. While Statement
5.1.1. Flowchart for while loop statement
5.1.2. While Loop Visualization


```
sum, current = 0, 1
n = 3
while current <= n:
    sum = sum + current
    current = current + 1
```

Tracking

Iteration	sum	current	current <= 3	sum = sum + current	current = current + 1
1	0	1	True	sum = 0 + 1 (1)	current = 1 + 1 (2)
2	1	2	True	sum = 1 + 2 (3)	current = 2 + 1 (3)
3	3	3	True	sum = 3 + 3 (6)	current = 3 + 1 (4)
4	6	4	False	loop termination	

While Loop – Counting Backward

Navigation

5. Iterative Control

5.1. While Statement

5,1,1, Flowchart for while loop statement

5.1.2. While Loop Visualization

5.1.3. While Loop – Counting Backward


```
import math

num = 25
while(num >= 10):
    print("The square root of", num, "is", math.sqrt(num))
    num-=5
```

```
The square root of 25 is 5.0
The square root of 20 is 4.47213595499958
The square root of 15 is 3.872983346207417
The square root of 10 is 3.1622776601683795
```

Here, we can see that, by importing math module, we are calculating square root of num variable i.e 25. With each iteration the count is decreasing by 5 (counting backward) using decrement variable

While Loop – Task Controlled

Navigation

5. Iterative Control

5.1. While Statement

5,1,1, Flowchart for while loop statement

5.1.2. While Loop Visualization

5.1.3. While Loop – Counting Backward

5.1.4. While Loop – Task Controlled


```
sum = 0
counter = 1
while (counter <= 10):
    sum = sum + counter
    counter = counter + 1
    print("The sum of numbers:", sum)

print("The sum of First 10 natural numbers:", sum)
```

```
The sum of numbers: 1
The sum of numbers: 3
The sum of numbers: 6
The sum of numbers: 10
The sum of numbers: 15
The sum of numbers: 21
The sum of numbers: 28
The sum of numbers: 36
The sum of numbers: 45
The sum of numbers: 55
```

```
The sum of First 10 natural numbers: 55
```

Here, we can see that, we can determine do we want to display the whole process during iteration or just the final result. With the help of sequence /(indentation) we can determine that

Before Tutorial!!

Navigation

5. Iterative Control

5.1. While Statement

5,1,1, Flowchart for while loop statement

5.1.2. While Loop Visualization

5.1.3. While Loop – Counting Backward

5.1.4. While Loop – Task Controlled

- Develop a system to find a factorial of n number, without using built in modules.

Can we check the error during input?

5.1.4. While Loop – Checking Error

Navigation

5. Iterative Control

5.1. While Statement

5,1,1, Flowchart for while loop statement

5.1.2. While Loop Visualization

5.1.3. While Loop – Counting Backward

5.1.4. While Loop – Checking Error

- The while statement is well suited for input error checking in a program.

What is input error checking and handling it?

- For instance, if we want the user to enter the even integer, we could have used the following code.

```
num = int(input("Please Enter The Number:"))
if (num%2==1):
    print("You Entered Odd Number, Please Enter The Value Again.")
else:
    print("The Number is Even")
```

It Identifies But!!

- That successfully identifies a mistake. But in order for error – checking to be truly helpful, we must provide the user the option to re-enter their information until it is accurate. This calls for repetition in general and a while loop in particular. To see an example, let's edit the above code.

Navigation

5. Iterative Control

5.1. While Statement

5,1,1, Flowchart for while loop statement

5.1.2. While Loop Visualization

5.1.3. While Loop – Counting Backward

5.1.4. While Loop – Checking Error continued

Let’s Fix This!!

Navigation

5. Iterative Control

5.1. While Statement

5,1,1, Flowchart for while loop statement

5.1.2. While Loop Visualization

5.1.3. While Loop – Counting Backward

5.1.4. While Loop – Checking Error

Fixing Code


```
num = int(input("Please Enter The Number:"))

while (num%2==1):
    print("You Entered Odd Number, Please Enter The Value Again.")
    num = int(input("Please Enter The Number:"))

print("The Number is Even")
```

```
Please Enter The Number:3
You Entered Odd Number, Please Enter The Value Again.
Please Enter The Number:2
The Number is Even
```

Here, we can see that when the user enters the odd number then, the system will prompt user to enter again.

Let's Try!!

Navigation

5. Iterative Control

5.1. While Statement

5,1,1, Flowchart for while loop statement

5.1.2. While Loop Visualization

5.1.3. While Loop – Counting Backward

5.1.4. While Loop – Checking Error

Code Tryouts

- Let's run this simple program in IDLE containing the code below and observe the result.

```
n = 10
sum = 0
current = 1

while current <= n:
        sum = sum + current
        current = current + 1
print(sum)
```

6. Sentinel Values

Navigation

1. Multiple Messages
2. Correct way to print
3. Solution for previous problem
4. Membership Operator
5. Iterative Control
6. Sentinel Values

- There are situation when input data endpoint is unknown. Using a sentinel value, the program can be instructed to stop accepting input.
- With the help of sentinel value we can simply terminate the loop
- If the input is from user, the user could enter the a specific value to stop collecting data.

[Scenario]: Imagine as a user you want a sum of certain positive integer but you don't know when to stop. Confused how to so? That's where we will be using sentinel value using in different type of loop

Let's Understand Through Example

Navigation

1. Multiple Messages
2. Correct way to print
3. Solution for previous problem
4. Membership Operator
5. Iterative Control
6. Sentinel Values

Example

```
num = int(input("Enter any number to add or 0 to stop "))
sum = 0

while(num!=0):
    sum = sum + num
    num = int(input("Enter any number to add or 0 to stop "))
print("The sum of numbers are: ", sum)
```

Loop condition will be true until the user's enter 0. Apart from 0, all the numbers entered by users will add and store it to a variable sum.

What’s Wrong With This Loop?

Navigation

1. Multiple Messages
2. Correct way to print
3. Solution for previous problem
4. Membership Operator
5. Iterative Control
6. Sentinel Values

Identifying problem with the code

```
n = 1
print('Loop starting..')
while n > 0:
        n = n + 1
        print('Count is {}'.format(n))
print('All Done!')
print('The final total is: {}'.format(n))
```

7. Infinite Loops

Navigation

1. Multiple Messages
2. Correct way to print
3. Solution for previous problem
4. Membership Operator
5. Iterative Control
6. Sentinel Values
7. Infinite Loops

- An infinite loop is an iterative control structure that never terminates (or eventually terminates with a system error).
- Infinite loops are generally the result of programming errors. For example, if the condition of a while loop can never be false, an infinite loop will result when executed.

However, they can be used to solve range problems

Example

Navigation

1. Multiple Messages
2. Correct way to print
3. Solution for previous problem
4. Membership Operator
5. Iterative Control
6. Sentinel Values
7. Infinite Loops

Example


```
n = 1
while n > 0:
    print(n)
```

8. Additional Iteration Control Statement

Navigation

1. Multiple Messages
2. Correct way to print
3. Solution for previous problem
4. Membership Operator
5. Iterative Control
6. Sentinel Values
7. Infinite Loops
8. Additional Iteration Control Statements

- There are two additional statements that we can use to help us write loops.

- **[Break Statement]:** The first is the break statement, which can be added to the code of block of a loop. When it is executed the current loop iteration is stopped and the loop is exited.

- [Continue Statement]: Execution then resumes with the continue statement that follows the loop statement.

8.1. Break Statement

Navigation

8. Additional Iteration Control Statements

8.1. Break Statement

- The break control statement gives us a way to end the execution of an infinite loop once a condition is met, making them much more useful.
- This sometimes allows us to cut down on repetitive code.
- However, using an infinite loop with the break or standard loop is mostly personal preference.

Example

```
i = 1
while (i<=10):
    print(i)
    i = i + 1
    if (i == 5):
        break
```

```
1
2
3
4
```

8.2. Continue Statement

Navigation

8. Additional Iteration Control Statements

8.1. Break Statement

8.2. Continue Statement

- The second additional statement that we can use to write loops is the **continue statement**.
- The **continue statement** can be added to a code block of a loop just like **break statement**.
- When the continue statement is executed the **current loop iteration is stopped and execution resumes with the next iteration**. Unlike the break statement, the continue statement does not terminate the loop. **It only terminates the current iteration.**

9. Definite VS. Indefinite Loops

Navigation

1. Multiple Messages
2. Correct way to print
3. Solution for previous problem
4. Membership Operator
5. Iterative Control
6. Sentinel Values
7. Infinite Loops
8. Additional Iteration Control Statements
9. Definite VS. Infinite Loops

- A **definite loop** is a program loop in which the number of times the loop will iterate can be determined before the loop is executed. Following is an example of definite loop.

```
sum = 0
current = 1
n = input('Enter Value ')
while current <= n:
        sum = sum + current
        current = current + 1
```

- **Although it is not known what the value of n will be until the input statement is executed, its value is known by the time the while is reached. Thus, it will execute n times.**

9.1. Indefinite Loops

Navigation

9. Definite VS. Infinite Loops

9.1. Indefinite Loops

- An **indefinite loop** is a program loop in which the number of times that the loop will iterate cannot be determined before the loop is executed. In this case, the number of times that the loop will be executed depends on how many times the user mistypes the input.

```
which = input('Enter Selection: ')
while which != 'F' or which != 'C':
        which = input("Please enter 'F' or 'C': ")
```

Guess The Topic!!

Navigation

9. Definite VS. Infinite Loops

9.1. Indefinite Loops

Guess the topic

9.2. Nested Loops

Navigation

9. Definite VS. Infinite Loops

9.1. Indefinite Loops

9.2. Nested Loops

- Like other control structures, loops in Python **can be nested inside other control structures**. You can nest a maximum of 20 control flow statements (more than enough).

```
while <condition>:
        while <condition>:
                statement(s)
        statement(s)
```

When is this practical? Oftentimes, one loop isn't enough, what if we wanted to:

- Write a program to print out a multiplication table??

Have Some Research!!

Before Tutorial Class, Have some research on!!

- For Loop Syntax
- When to use which loop?
- Range Function

Navigation

1. Multiple Messages
2. Correct way to print
3. Solution for previous problem
4. Membership Operator
5. Iterative Control
6. Sentinel Values
7. Infinite Loops
8. Additional Iteration Control Statements
9. Definite VS. Infinite Loops

UNIVERSITY OF WOLVERHAMPTON

BIRATNAGAR INTERNATIONAL COLLEGE

ing